

NEXAANOVA CRM SYSTEM

Software Development Documentation

Student Project Submission

Name:	Aaryaa Tugaonkar
FRN:	02J0126/022 (JAVA 16 JAN)
Project:	NEXAANOVA CRM System
Domain:	Education / Admissions Management
Technology:	Web-based CRM Application
Year:	2026

1. SYNOPSIS

1.1 Project Overview

NEXAANOVA CRM is a web-based Customer Relationship Management system designed specifically for educational institutions to streamline the student admission process. The system provides a centralized platform to manage student enquiries, track counselor interactions, schedule follow-ups, and record admissions — improving efficiency and transparency across the admission pipeline.

1.2 Problem Statement

Educational institutions face significant challenges in managing student enquiries manually. Common issues include:

- Loss of lead data due to unstructured record-keeping
- Missed follow-ups causing student drop-off
- Inability to track counselor performance
- No centralized system for admission conversion tracking
- Lack of real-time reporting for management decisions

1.3 Proposed Solution

The NEXAANOVA CRM system addresses these issues by providing:

- A centralized digital repository for all student enquiries
- Role-based access for Admin, Counselors, and Manager
- Automated follow-up scheduling with reminders
- Structured call tracking with status management
- Seamless enquiry-to-admission conversion workflow
- Comprehensive reporting and analytics dashboard

1.4 Project Scope

The scope of the NEXAANOVA CRM includes:

1. Login & User Management Module
2. Enquiry Management Module
3. Calling Module
4. Follow-up Module
5. Admission Module
6. Course Management Module
7. Reports & Analytics Module

1.5 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript / React	User Interface
Backend	Java (Spring Boot), Hibernate/JPA, Maven	Business Logic & ORM
Database	MySQL / PostgreSQL	Data Storage
Server	Apache Tomcat / Nginx	Web Hosting

1.6 Key Benefits

- Improved lead conversion rates through systematic follow-up management
- Real-time visibility into admission pipeline for management
- Reduced manual effort and human error
- Performance tracking for counselors
- Scalable and extensible architecture

1.7 User Roles Summary

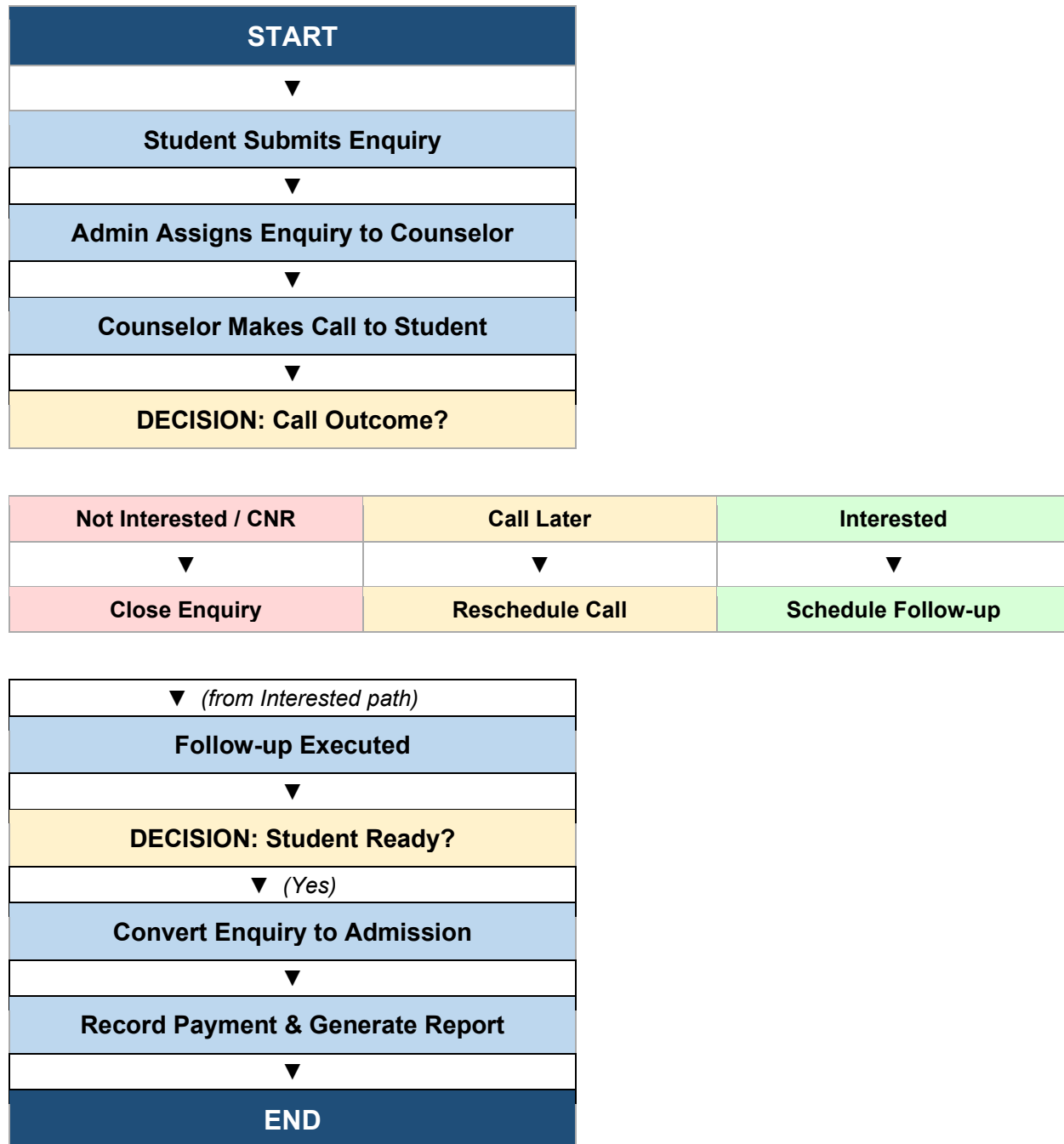
Role	Responsibilities	Access Level
Admin	User management, enquiry assignment, course/settings management	Full Access
Counselor	View enquiries, make calls, schedule follow-ups, convert to admissions	Assigned Enquiries
Manager	View team performance, access reports and dashboards	Read-only Reports

2. FLOW DIAGRAM

2.1 System Process Flow

The NEXAANOVA CRM follows a structured process flow from the moment a student enquiry enters the system until the student is admitted or the enquiry is closed.

The main process flow is represented as follows:



2.2 Module Interaction Flow

The following table describes how each module interacts within the system:

Module	Input	Output
Enquiry Module	Student details from walk-in/web/call	Enquiry record created, assigned to counselor
Calling Module	Counselor call attempt on enquiry	Call status logged, next action triggered
Follow-up Module	Interested student, follow-up date set	Reminder sent, follow-up result logged
Admission Module	Confirmed student, course and fee details	Admission record, payment receipt generated
Reports Module	Date range, filter criteria	Analytics dashboard, exportable reports

3. ACTIVITY DIAGRAMS

Activity diagrams illustrate the dynamic behavior of each module within the NEXAANOVA CRM system, showing the sequence of actions and decision points for each process.

3.1 User Login & Role Management

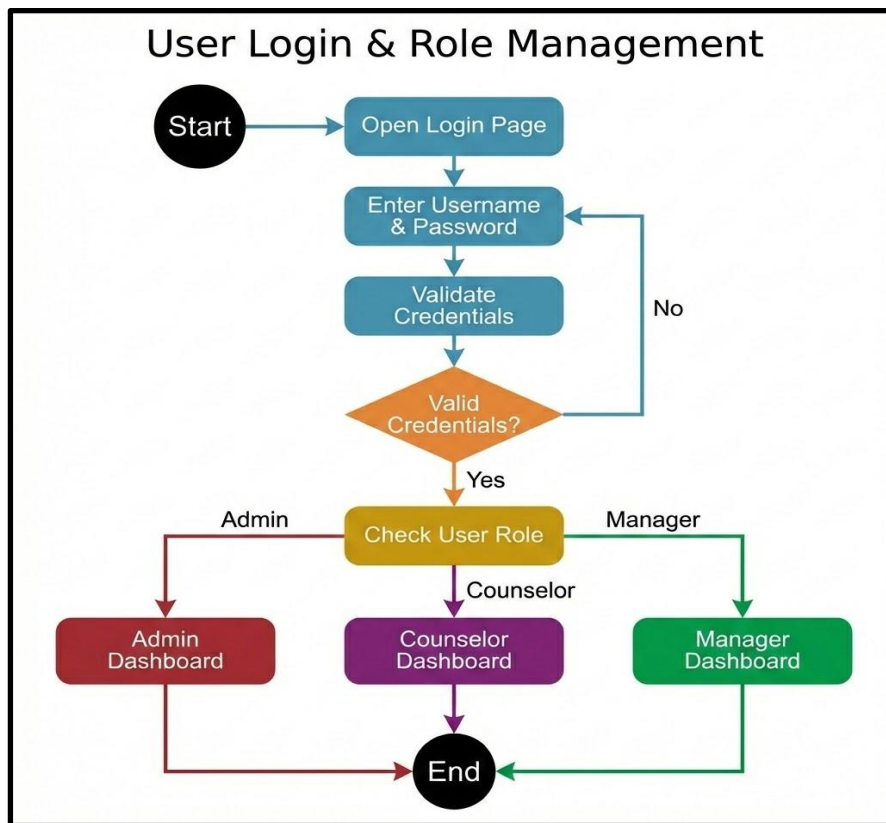
Purpose: Shows how different users access the system with role-based dashboards.

Actors:

- Admin
- Counselor
- Manager

Activity Steps:

Step	Activity / Decision	Outcome
1	User opens CRM login page	Login page displayed
2	User enters username and password	Credentials captured
3	System validates credentials against database	Validation check initiated
4	DECISION: Are credentials valid?	If No → Show error message, return to Step 1
5	System fetches user role from database	Role identified
6	DECISION: Role = Admin?	If Yes → Open Admin Dashboard
7	DECISION: Role = Manager?	If Yes → Open Manager Dashboard
8	DECISION: Role = Counselor?	If Yes → Open Counselor Dashboard
9	User begins session in role-based dashboard	Session started, logged in



3.2 Enquiry Creation

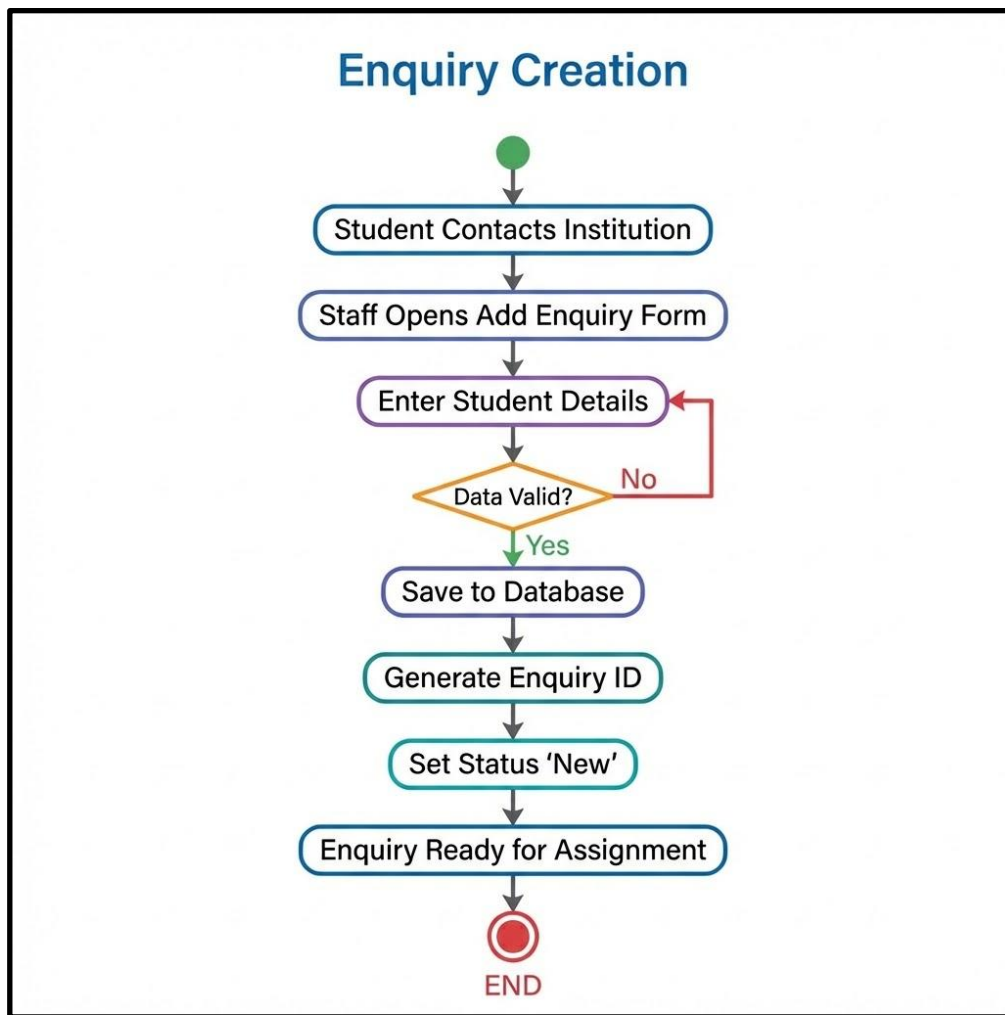
Purpose: Shows how a new student lead is captured and entered into the CRM.

Actors:

- Staff / Counselor
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	Student contacts institution (walk-in, call, web, Instagram)	Enquiry source identified
2	Staff opens 'Add Enquiry' form in CRM	New enquiry form displayed
3	Staff enters student details (name, phone, email, city, course, source)	Form fields populated
4	DECISION: Is all required data valid?	If No → Show validation errors
5	System saves enquiry to database	Enquiry record created
6	System auto-generates unique Enquiry ID	Enquiry ID assigned
7	System sets status to 'New'	Enquiry status initialized
8	Enquiry appears in admin assignment queue	Ready for assignment



3.3 Enquiry Assignment

Purpose: Shows how enquiries are assigned to counselors by the admin.

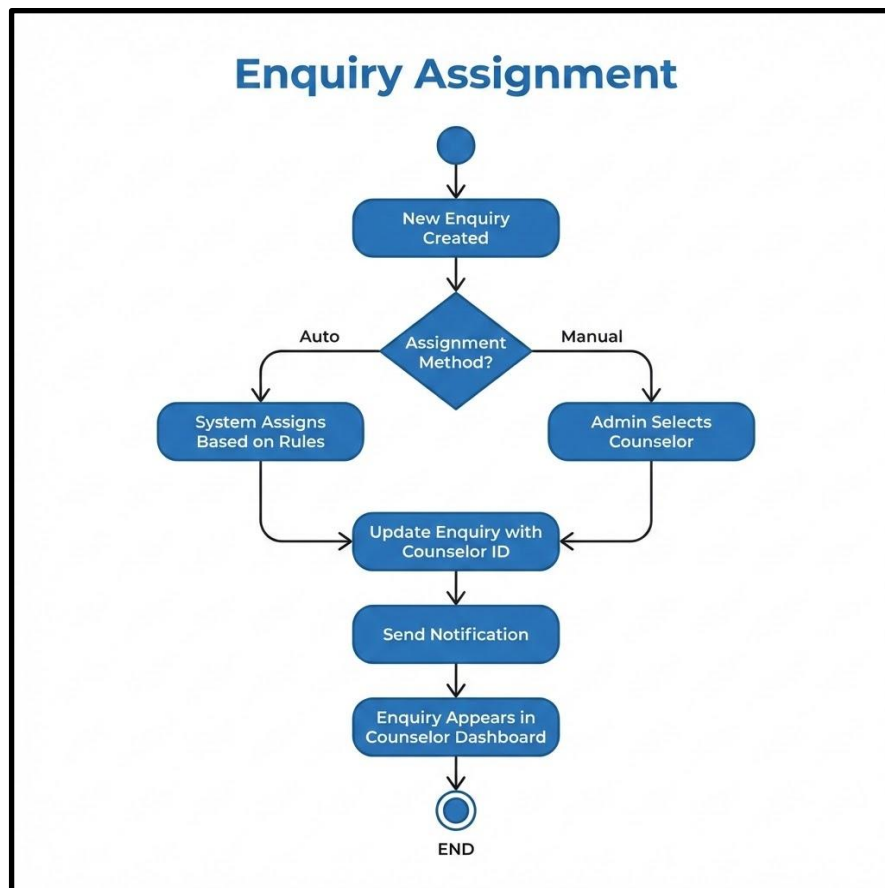
Actors:

- Admin
- System
- Counselor

Activity Steps:

Step	Activity / Decision	Outcome
1	New enquiry created with status 'New'	Enquiry in queue
2	Admin opens assignment dashboard	Unassigned enquiries visible
3	DECISION: Assignment method?	Auto-assign or Manual-assign selected
4	Manual: Admin selects specific counselor from list	Counselor chosen
5	Auto: System assigns based on workload/rules	Counselor auto-assigned

6	System updates enquiry with assigned counselor ID	Assignment saved
7	System sends notification to counselor	Counselor notified
8	Enquiry appears in counselor's dashboard	Counselor receives lead



3.4 Calling Process

Purpose: Shows what happens during a counselor's call to a student.

Actors:

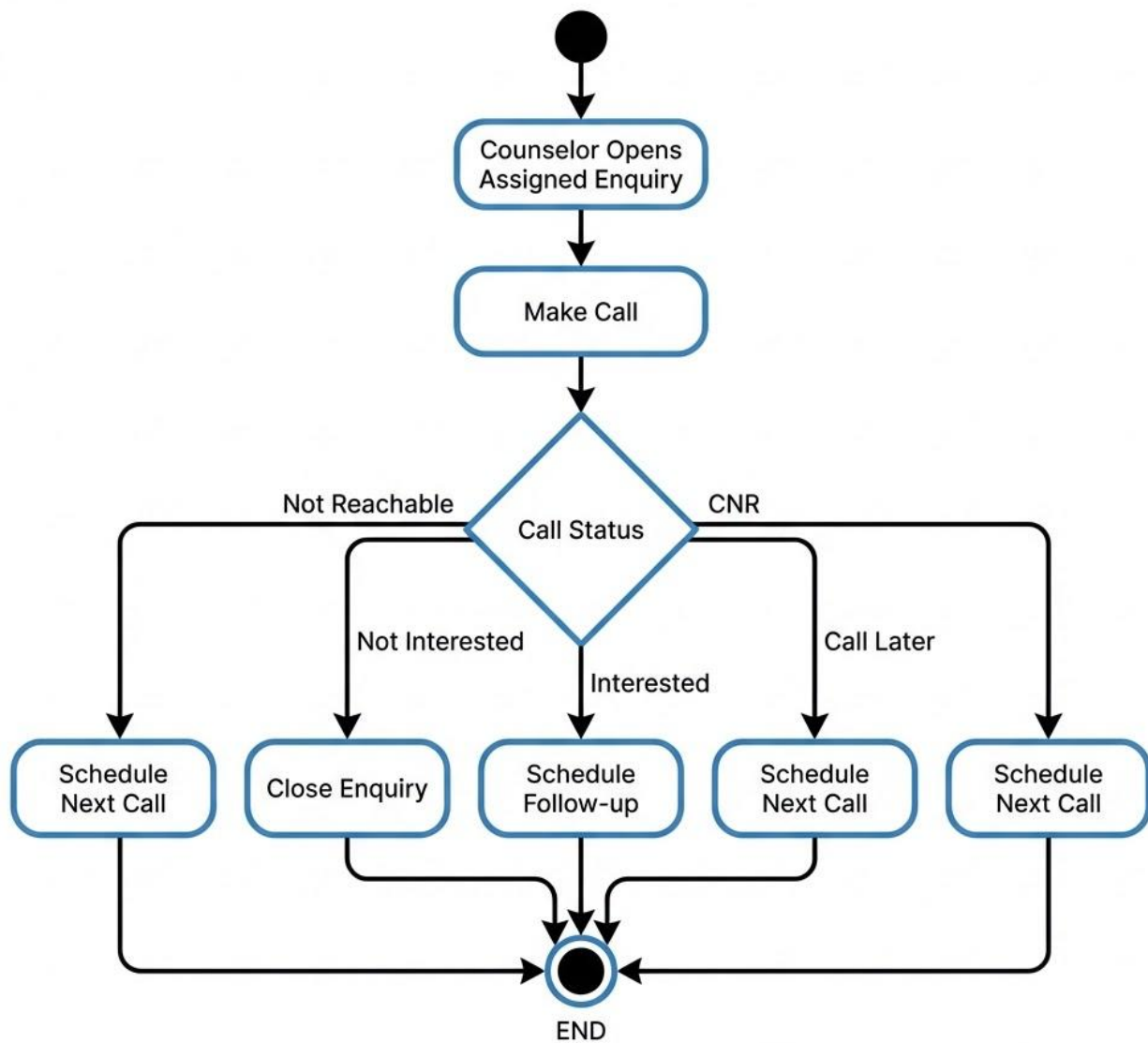
- Counselor
- Student

Activity Steps:

Step	Activity / Decision	Outcome
1	Counselor logs in and opens assigned enquiries	Enquiry list visible
2	Counselor selects enquiry and views student details	Student info displayed
3	Counselor places call to student	Call initiated
4	DECISION: Is student reachable?	If No → Set status 'Not Reachable / CNR', schedule next call

5	DECISION: Is student interested?	If No → Set status 'Not Interested', close enquiry
6	Counselor enters call remarks and updates status to 'Interested'	Call record saved
7	System prompts to schedule follow-up	Follow-up scheduling triggered
8	Call record saved with timestamp	Calling history updated

Calling Process



3.5 Follow-up Scheduling

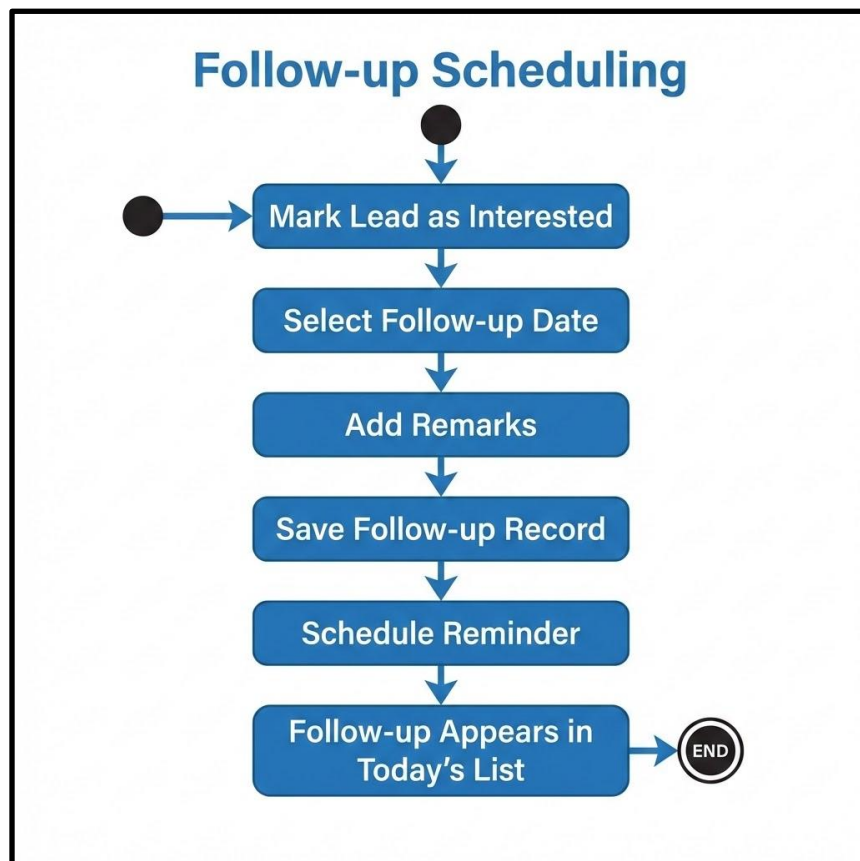
Purpose: Shows how follow-up tasks are planned for interested students.

Actors:

- Counselor
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	Counselor marks student as 'Interested' after call	Interest recorded
2	System prompts to create follow-up task	Follow-up form shown
3	Counselor selects follow-up date and time	Date/time captured
4	Counselor adds remarks and follow-up notes	Notes saved
5	System saves follow-up record linked to Enquiry ID	Follow-up created
6	System schedules automated reminder	Reminder set for follow-up date
7	Follow-up task appears in counselor's today's list on due date	Task visible



3.6 Follow-up Execution

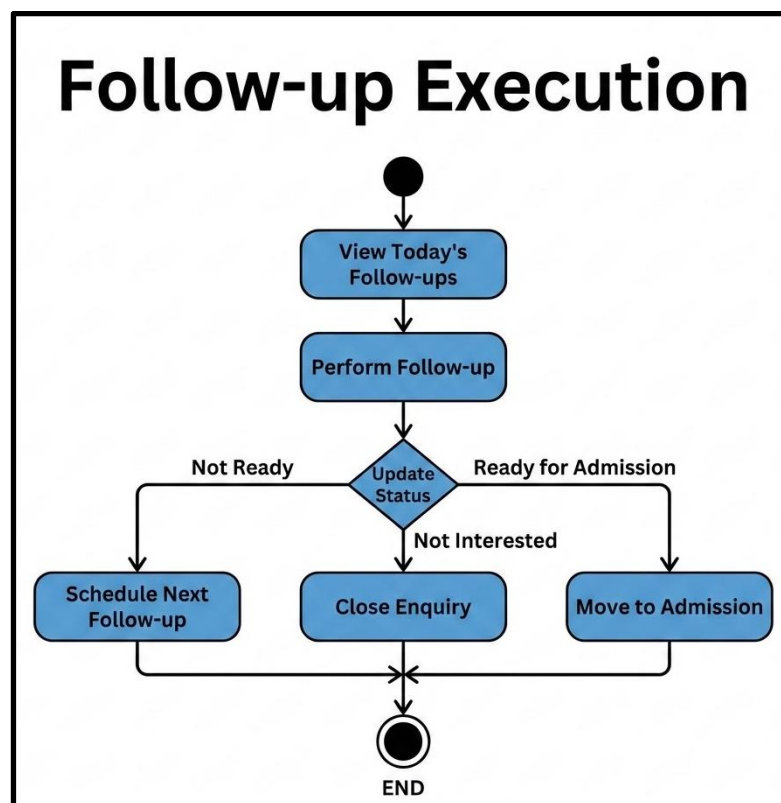
Purpose: Shows how follow-ups are handled on the scheduled date.

Actors:

- Counselor
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	System displays today's follow-ups on counselor dashboard	Due follow-ups listed
2	Counselor opens follow-up task and reviews notes	Context reviewed
3	Counselor contacts student for follow-up	Follow-up executed
4	DECISION: Is student ready to admit?	If Yes → Move to admission stage
5	DECISION: Is student still interested but not ready?	If Yes → Schedule next follow-up
6	DECISION: Is student not interested?	If Yes → Close enquiry
7	Counselor updates follow-up status and adds remarks	Follow-up record updated
8	Enquiry status updated accordingly	Lead pipeline refreshed



3.7 Admission Conversion

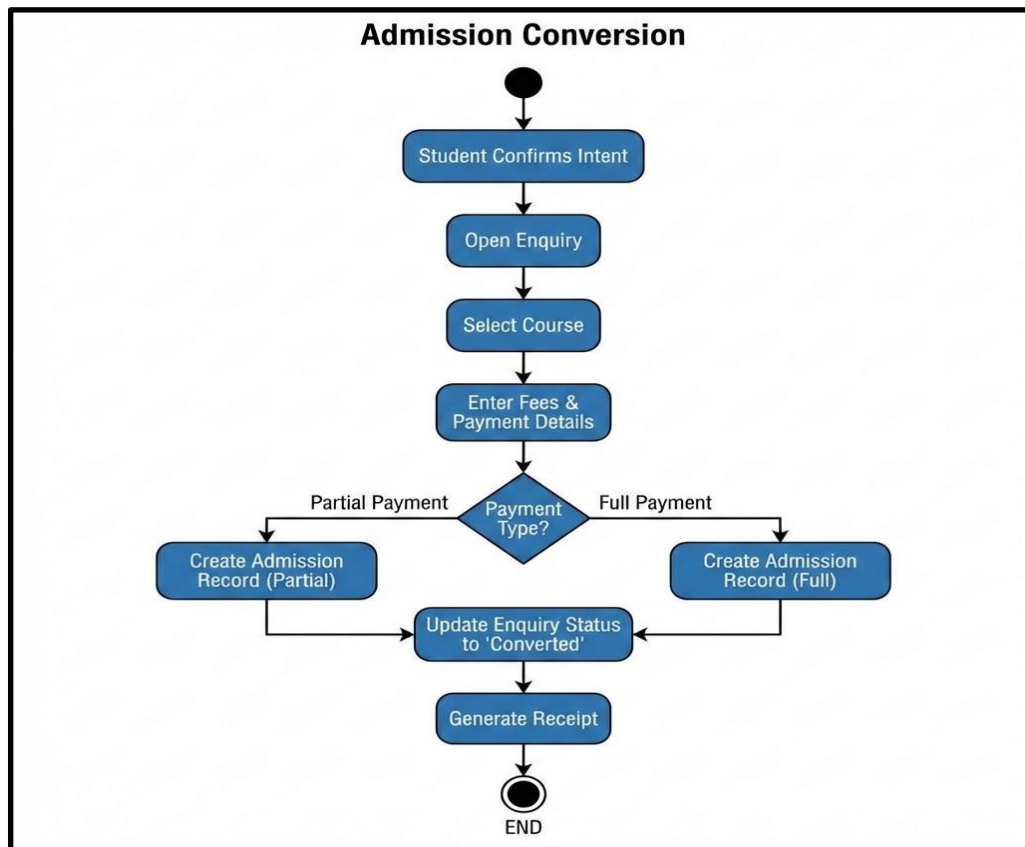
Purpose: Shows conversion of a confirmed student lead into an admission record.

Actors:

- Counselor
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	Student confirms intent to join	Admission trigger
2	Counselor opens enquiry and selects 'Convert to Admission'	Conversion form opened
3	Counselor selects course for student	Course linked
4	Counselor enters total fees and fees paid	Fee data entered
5	Counselor selects payment type (Cash/Online/Cheque)	Payment mode captured
6	DECISION: Full or Partial payment?	If Partial → Create installment schedule
7	System creates admission record with unique Admission ID	Admission saved
8	Enquiry status changed to 'Converted'	Lead closed as converted
9	System generates admission receipt	Document available for download



3.8 Payment Handling

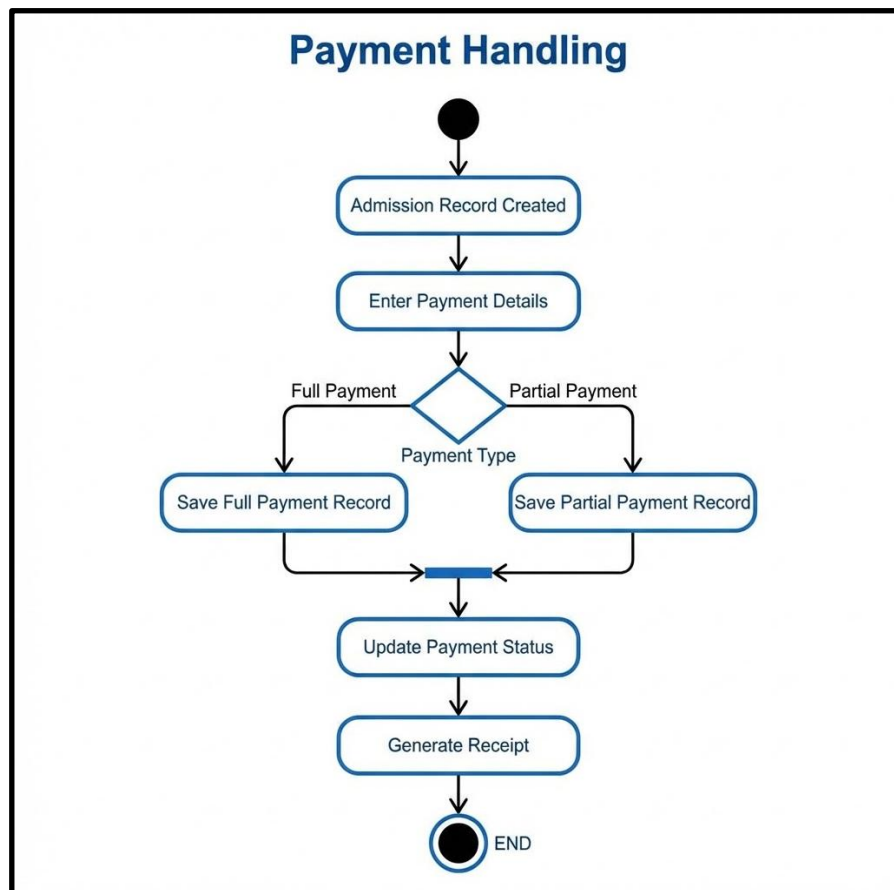
Purpose: Shows how fee payments are recorded and tracked.

Actors:

- Counselor
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	Admission record created	Payment recording started
2	Counselor opens payment section of admission	Payment form displayed
3	Counselor enters payment amount and date	Amount captured
4	Counselor selects payment mode (Cash/Online/Cheque/DD)	Mode selected
5	DECISION: Is total fees fully paid?	If Yes → Mark status 'Paid'
6	DECISION: Is amount partial?	If Yes → Mark status 'Pending Balance'
7	System calculates remaining balance if partial	Balance computed
8	System saves payment record	Transaction saved
9	System generates payment receipt	Receipt available



3.9 Enquiry Closure

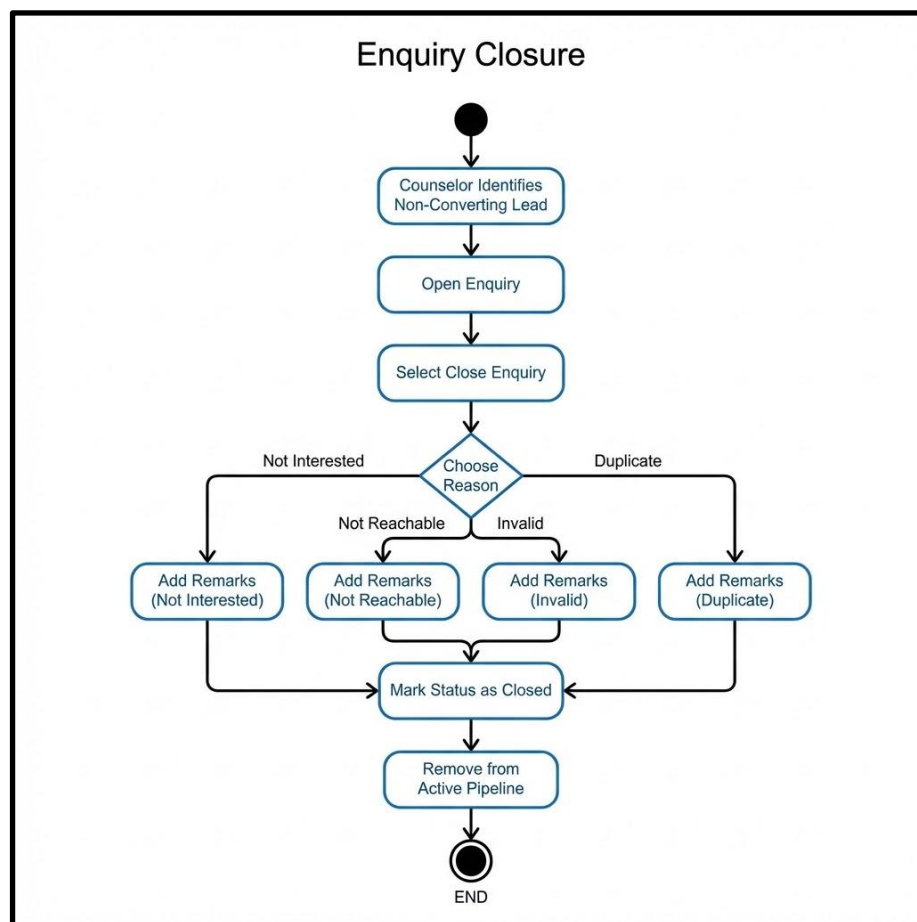
Purpose: Shows how non-converting leads are closed and archived.

Actors:

- Counselor
- Admin

Activity Steps:

Step	Activity / Decision	Outcome
1	Counselor identifies lead as non-converting	Closure initiated
2	Counselor opens enquiry and selects 'Close Enquiry'	Closure form shown
3	Counselor selects closure reason	Reason options: Not Interested / Not Reachable / Invalid / Duplicate
4	Counselor adds final remarks	Notes recorded
5	System marks enquiry status as 'Closed'	Status updated
6	Enquiry removed from active pipeline	Pipeline cleaned
7	Closure data available for reports	Analytics updated



3.10 Report Generation

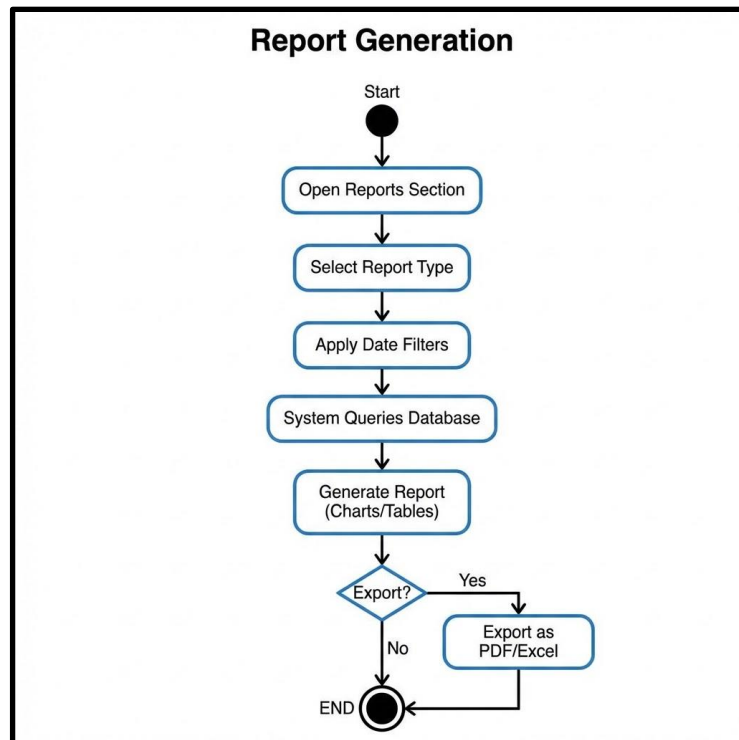
Purpose: Shows how management reports are generated from system data.

Actors:

- Admin
- Manager
- System

Activity Steps:

Step	Activity / Decision	Outcome
1	User navigates to Reports section	Report dashboard opened
2	User selects report type	Options: Enquiry / Follow-up / Admission / Counselor Performance
3	User applies filters (date range, counselor, course)	Filter criteria set
4	System queries database with parameters	Data fetched
5	System aggregates and calculates metrics	Analytics computed
6	System displays report with charts/tables	Report rendered
7	DECISION: Export required?	If Yes → Export as PDF or Excel
8	Report saved or shared	Task complete



3.11 Course Management

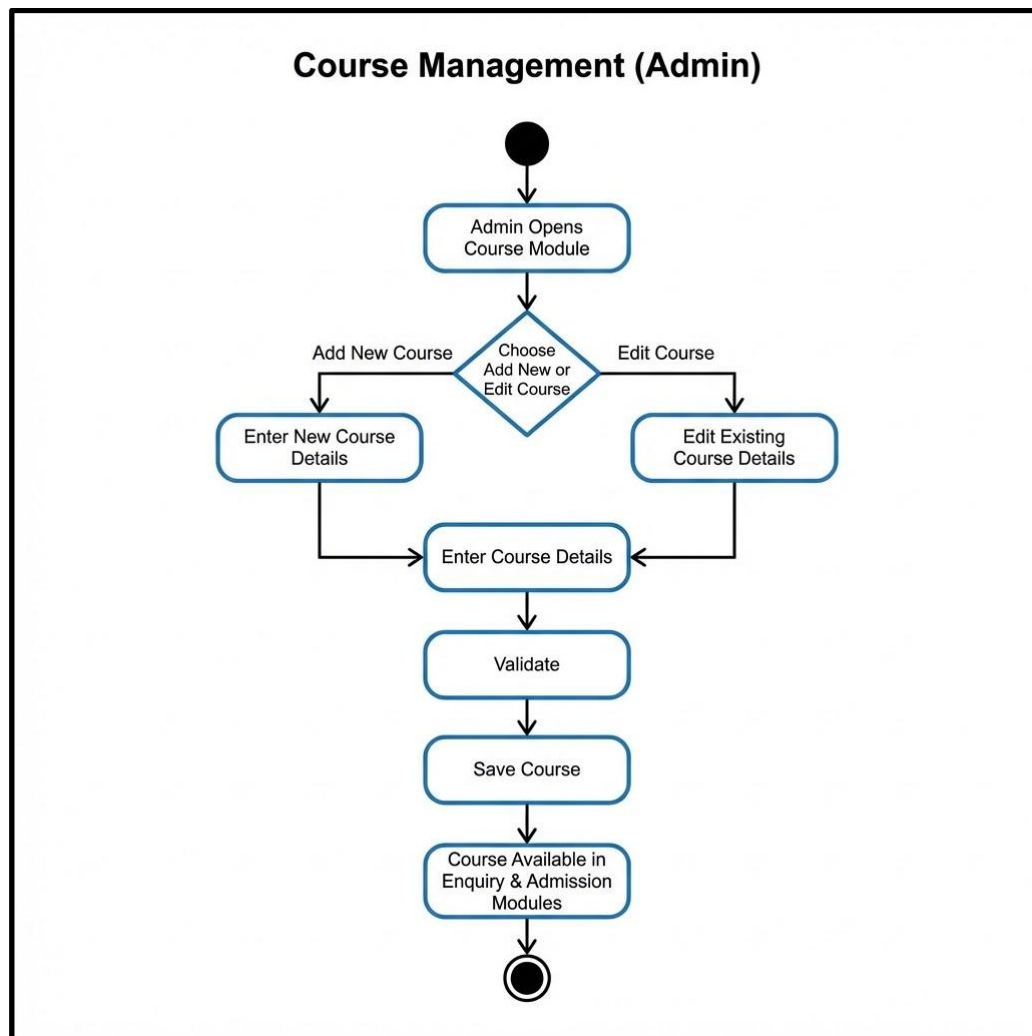
Purpose: Shows how admin manages course catalog in the system.

Actors:

- Admin

Activity Steps:

Step	Activity / Decision	Outcome
1	Admin navigates to Course Management module	Course list displayed
2	Admin selects 'Add New Course' or selects existing course	Action chosen
3	Admin enters course name, description, duration, fees	Course details filled
4	DECISION: Is course new or edit?	If New → Create record; If Edit → Update record
5	System validates and saves course data	Course saved
6	Course becomes available in Enquiry and Admission modules	Linked to other modules



3.12 User Management (Admin)

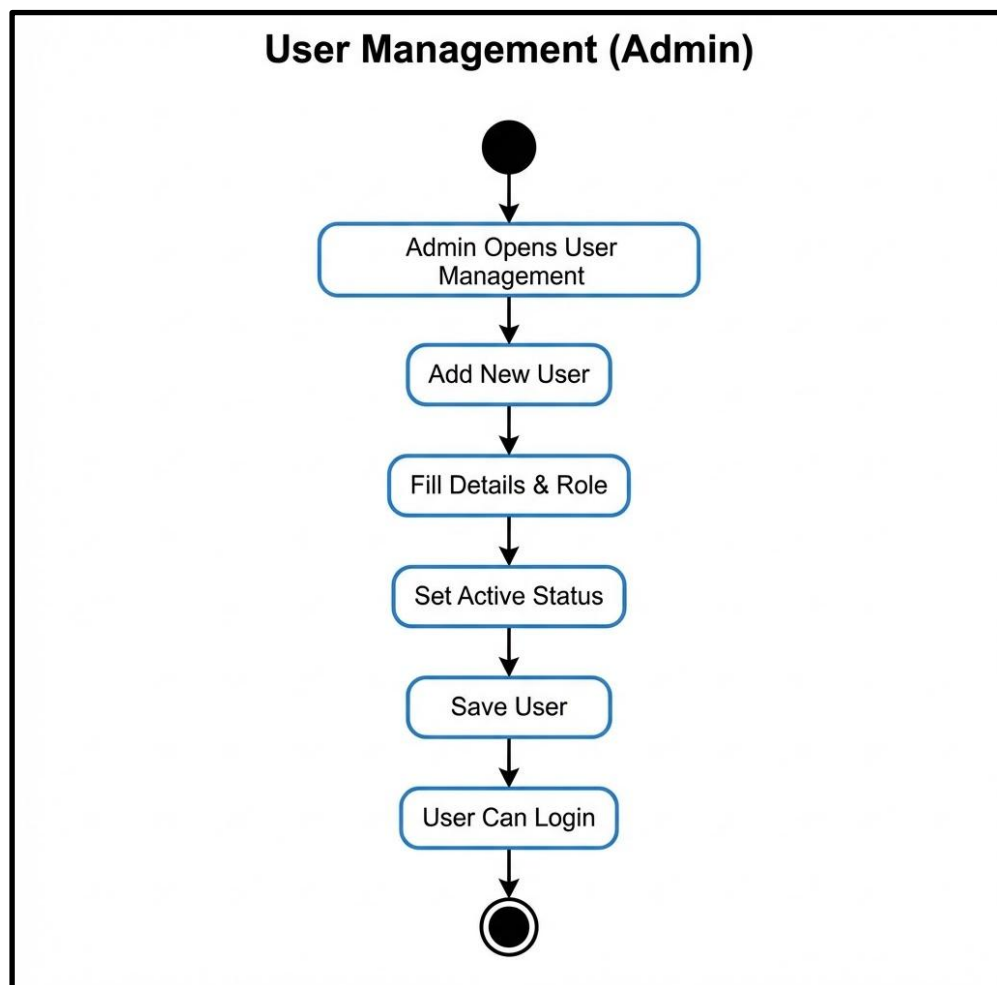
Purpose: Shows how admin creates and manages system users.

Actors:

- Admin
- System

Activity Steps:

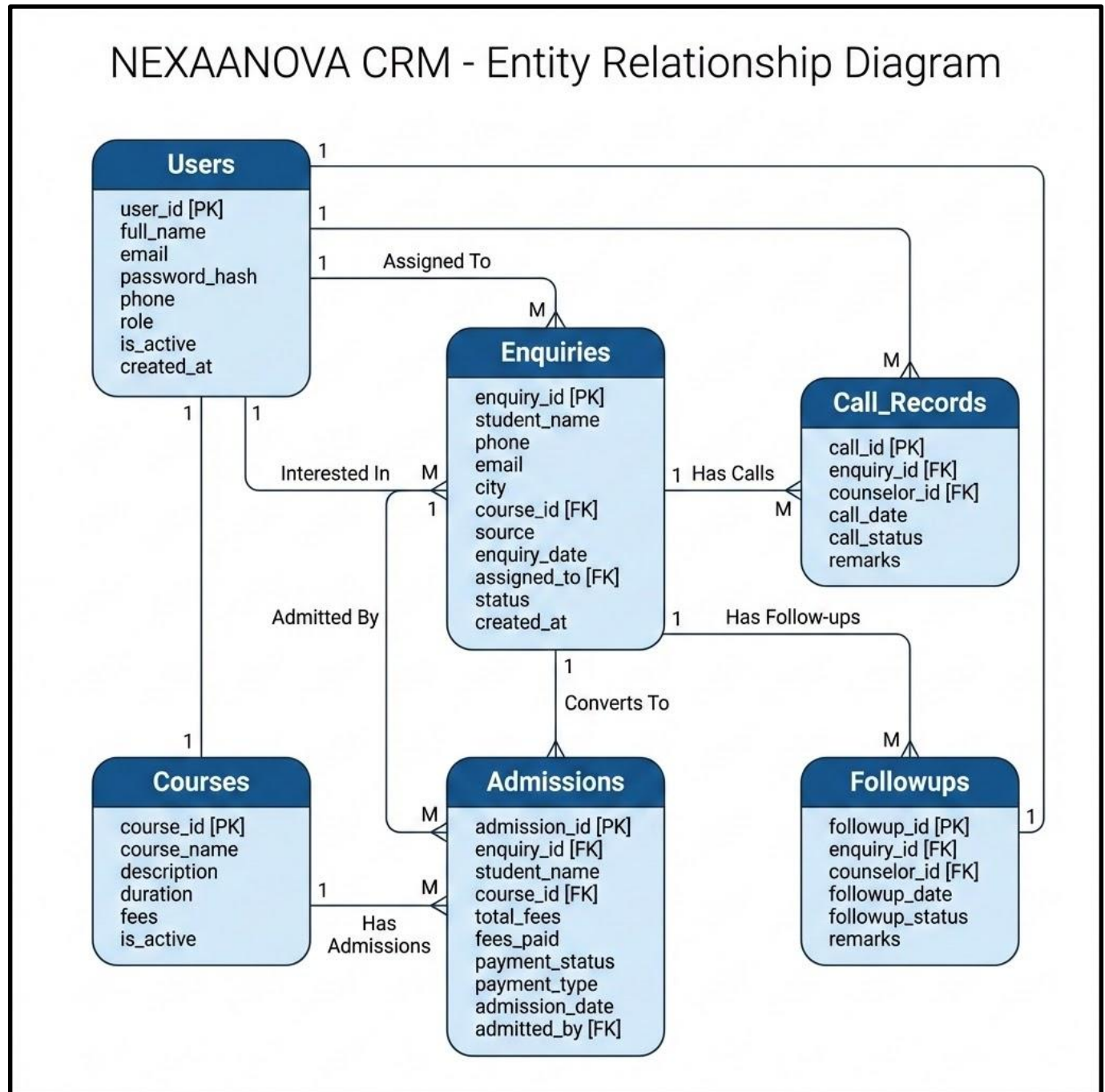
Step	Activity / Decision	Outcome
1	Admin logs in and navigates to User Management	User list displayed
2	Admin selects 'Add New User'	User creation form shown
3	Admin enters name, email, phone, password	User details filled
4	Admin assigns role (Admin / Counselor / Manager)	Role selected
5	Admin sets user status (Active / Inactive)	Status configured
6	System saves user record to database	User account created
7	New user can now log in with assigned credentials	Access granted



4. ER DIAGRAM

4.1 Overview

The Entity-Relationship (ER) Diagram describes the data model for the NEXAANOVA CRM system. It shows all entities (tables), their attributes (fields), and the relationships between them.



4.2 Entities and Attributes

4.2.1 Users

Attribute	Data Type	Constraint	Description
user_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	Full name of user
email	VARCHAR(150)	UNIQUE, NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	Encrypted password
phone	VARCHAR(15)	NULL	Contact number
role	ENUM('admin','counselor','manager')	NOT NULL	Access role
is_active	BOOLEAN	DEFAULT TRUE	Account status
created_at	TIMESTAMP	DEFAULT NOW()	Account creation date

4.2.2 Enquiries

Attribute	Data Type	Constraint	Description
enquiry_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Auto-generated ID
student_name	VARCHAR(100)	NOT NULL	Student full name
phone	VARCHAR(15)	NOT NULL	Contact number
email	VARCHAR(150)	NULL	Student email
city	VARCHAR(100)	NULL	Student city
course_id (FK)	INT	FOREIGN KEY	Course of interest
source	ENUM('Instagram','Website','Walk-in','Reference','Other')	NOT NULL	Enquiry source
enquiry_date	DATE	DEFAULT NOW()	Date enquiry created
assigned_to (FK)	INT	FOREIGN KEY → users	Assigned counselor
status	ENUM('New','Called','Follow-up','Closed','Converted','CNR')	DEFAULT 'New'	Current status

4.2.3 Call_Records

Attribute	Data Type	Constraint	Description
call_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Call record ID
enquiry_id (FK)	INT	FOREIGN KEY → enquiries	Related enquiry
counselor_id (FK)	INT	FOREIGN KEY → users	Counselor who called
call_date	DATETIME	NOT NULL	Date and time of call
call_status	ENUM('Not Reachable','Not Interested','Interested','Call Later','CNR')	NOT NULL	Outcome of call
remarks	TEXT	NULL	Call notes

4.2.4 Followups

Attribute	Data Type	Constraint	Description
followup_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Follow-up record ID
enquiry_id (FK)	INT	FOREIGN KEY → enquiries	Related enquiry
counselor_id (FK)	INT	FOREIGN KEY → users	Assigned counselor
followup_date	DATE	NOT NULL	Scheduled date
followup_status	ENUM('Pending','Done','Rescheduled','Converted','Closed')	DEFAULT 'Pending'	Follow-up outcome
remarks	TEXT	NULL	Follow-up notes

4.2.5 Admissions

Attribute	Data Type	Constraint	Description
admission_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Admission record ID
enquiry_id (FK)	INT	FOREIGN KEY → enquiries	Source enquiry
student_name	VARCHAR(100)	NOT NULL	Student full name

course_id (FK)	INT	FOREIGN KEY → courses	Enrolled course
total_fees	DECIMAL(10,2)	NOT NULL	Total course fees
fees_paid	DECIMAL(10,2)	NOT NULL	Amount paid so far
payment_status	ENUM('Paid','Pending','Installment')	NOT NULL	Payment status
payment_type	ENUM('Cash','Online','Cheque','DD')	NOT NULL	Payment method
admission_date	DATE	DEFAULT NOW()	Date of admission
admitted_by (FK)	INT	FOREIGN KEY → users	Counselor who admitted

4.2.6 Courses

Attribute	Data Type	Constraint	Description
course_id (PK)	INT AUTO_INCREMENT	PRIMARY KEY	Course identifier
course_name	VARCHAR(150)	NOT NULL, UNIQUE	Name of course
description	TEXT	NULL	Course description
duration	VARCHAR(50)	NOT NULL	Duration (e.g., 6 months)
fees	DECIMAL(10,2)	NOT NULL	Standard course fees
is_active	BOOLEAN	DEFAULT TRUE	Active/Inactive status

4.3 Relationships

From Entity	Relationship	To Entity	Description
Users	1 to Many	Enquiries	One counselor handles many enquiries
Enquiries	1 to Many	Call_Records	One enquiry can have multiple call records
Enquiries	1 to Many	Followups	One enquiry can have multiple follow-ups
Enquiries	1 to 1	Admissions	One enquiry converts to one admission
Courses	1 to Many	Enquiries	One course linked to many enquiries
Courses	1 to Many	Admissions	One course has many admissions
Users	1 to Many	Admissions	One counselor can record many admissions

5. SEQUENCE DIAGRAM

Sequence diagrams show the interaction between system actors and components over time. They describe the order of messages exchanged to complete a use case.

5.1 Sequence: User Login

Seq	User/Browser	Login Controller	Auth Service	Database	Response
1	Enter credentials → Submit	Receive login request	—	—	—
2	—	Call AuthService.validate()	Receive credentials	—	—
3	—	—	Query user by email	Return user record	—
4	—	—	Verify password hash	—	—
5	—	Get role, create session	Return valid/invalid	—	—
6	Redirect to dashboard	Return dashboard URL	—	—	Dashboard shown

5.2 Sequence: Add Enquiry

Seq	Counselor/Admin	Enquiry Controller	Validation Service	Database	Response
1	Fill & submit form	Receive form data	—	—	—
2	—	Call validate(data)	Check required fields	—	—
3	—	If valid: save enquiry	Return OK/Error	INSERT enquiry row	—
4	—	—	—	Return new enquiry_id	—
5	Show success message	Return success + ID	—	—	Enquiry created

5.3 Sequence: Call Record Entry

Seq	Counselor	Call Controller	Enquiry Service	Database	Response
1	Open enquiry, log call	Receive call data	—	—	—
2	—	INSERT call record	—	Save call row	—
3	—	Update enquiry status	UPDATE status field	Status updated	—
4	—	If interested: prompt follow-up	—	—	—
5	Call logged	Return confirmation	—	—	Call history shown

5.4 Sequence: Admission Conversion

Seq	Counselor	Admission Controller	Enquiry Service	Database	Response
1	Select 'Convert to Admission'	Open admission form	—	—	—
2	Fill course, fees, payment	Validate & save admission	—	INSERT admission row	—
3	—	Update enquiry status	SET status='Converted'	Enquiry updated	—
4	—	Generate receipt	—	Return admission_id	—
5	Receipt downloaded	Return receipt PDF link	—	—	Admission complete

6. DATABASE DESIGN

6.1 Database Overview

The NEXAANOVA CRM uses a relational database management system (RDBMS) such as MySQL or PostgreSQL. The database consists of 6 core tables with well-defined relationships using foreign keys.

6.2 SQL Table Definitions

Table: users

```
CREATE TABLE users (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  phone        VARCHAR(15),
  role         ENUM('admin','counselor','manager') NOT NULL,
  is_active    BOOLEAN DEFAULT TRUE,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
// JPA Entity Equivalent (Spring Boot)
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;

    @Column(nullable = false, length = 100)
    private String fullName;

    @Column(unique = true, nullable = false, length = 150)
    private String email;

    @Column(nullable = false, length = 255)
    private String passwordHash;

    @Column(length = 15)
    private String phone;

    @Enumerated(EnumType.STRING)
    private Role role;

    private Boolean isActive = true;

    @CreationTimestamp
    private Timestamp createdAt;

    // Getters, Setters, Constructors
}
```

Table: courses

```
CREATE TABLE courses (  
  course_id    INT AUTO_INCREMENT PRIMARY KEY,  
  course_name  VARCHAR(150) NOT NULL UNIQUE,  
  description  TEXT,  
  duration     VARCHAR(50) NOT NULL,  
  fees         DECIMAL(10,2) NOT NULL,  
  is_active    BOOLEAN DEFAULT TRUE  
);
```

```
// JPA Entity Equivalent (Spring Boot)  
@Entity  
@Table(name = "courses")  
public class Course {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long courseId;  
  
    @Column(nullable = false, unique = true, length = 150)  
    private String courseName;  
  
    @Column(columnDefinition = "TEXT")  
    private String description;  
  
    @Column(nullable = false, length = 50)  
    private String duration;  
  
    @Column(nullable = false, precision = 10, scale = 2)  
    private BigDecimal fees;  
  
    private Boolean isActive = true;  
  
    // Getters, Setters, Constructors  
}
```

Table: enquiries

```
CREATE TABLE enquiries (  
  enquiry_id   INT AUTO_INCREMENT PRIMARY KEY,  
  student_name VARCHAR(100) NOT NULL,  
  phone        VARCHAR(15) NOT NULL,  
  email        VARCHAR(150),  
  city         VARCHAR(100),  
  course_id    INT, FOREIGN KEY (course_id) REFERENCES courses(course_id),  
  source       ENUM('Instagram','Website','Walk-in','Reference','Other')  
NOT NULL,  
  enquiry_date DATE DEFAULT (CURRENT_DATE),  
  assigned_to  INT, FOREIGN KEY (assigned_to) REFERENCES users(user_id),  
  status       ENUM('New','Called','Follow-up','Closed','Converted','CNR')  
DEFAULT 'New',  
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
// JPA Entity Equivalent (Spring Boot)  
@Entity  
@Table(name = "enquiries")  
public class Enquiry {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long enquiryId;  
  
    @Column(nullable = false, length = 100)
```

```

private String studentName;

@Column(nullable = false, length = 15)
private String phone;

@Column(length = 150)
private String email;

@Column(length = 100)
private String city;

@ManyToOne
@JoinColumn(name = "course_id")
private Course course;

@Enumerated(EnumType.STRING)
private Source source;

@Enumerated(EnumType.STRING)
private EnquiryStatus status = EnquiryStatus.NEW;

@ManyToOne
@JoinColumn(name = "assigned_to")
private User assignedTo;

@CreationTimestamp
private Timestamp createdAt;

// Getters, Setters, Constructors
}

```

Table: call_records

```

CREATE TABLE call_records (
  call_id      INT AUTO_INCREMENT PRIMARY KEY,
  enquiry_id   INT NOT NULL, FOREIGN KEY (enquiry_id) REFERENCES
enquiries(enquiry_id),
  counselor_id INT NOT NULL, FOREIGN KEY (counselor_id) REFERENCES
users(user_id),
  call_date    DATETIME NOT NULL,
  call_status  ENUM('Not Reachable','Not Interested','Interested','Call
Later','CNR') NOT NULL,
  remarks      TEXT
);

```

```

// JPA Entity Equivalent (Spring Boot)
@Entity
@Table(name = "call_records")
public class CallRecord {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long callId;

  @ManyToOne
  @JoinColumn(name = "enquiry_id", nullable = false)
  private Enquiry enquiry;

  @ManyToOne
  @JoinColumn(name = "counselor_id", nullable = false)

```

```
private User counselor;

@Column(nullable = false)
private LocalDateTime callDate;

@Enumerated(EnumType.STRING)
private CallStatus callStatus;

@Column(columnDefinition = "TEXT")
private String remarks;

// Getters, Setters, Constructors
}
```

Table: followups

```
CREATE TABLE followups (
  followup_id      INT AUTO_INCREMENT PRIMARY KEY,
  enquiry_id       INT NOT NULL, FOREIGN KEY (enquiry_id) REFERENCES
enquiries(enquiry_id),
  counselor_id     INT NOT NULL, FOREIGN KEY (counselor_id) REFERENCES
users(user_id),
  followup_date    DATE NOT NULL,
  followup_status  ENUM('Pending','Done','Rescheduled','Converted','Closed')
DEFAULT 'Pending',
  remarks          TEXT
);
```

```
// JPA Entity Equivalent (Spring Boot)
@Entity
@Table(name = "followups")
public class Followup {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long followupId;

    @ManyToOne
    @JoinColumn(name = "enquiry_id", nullable = false)
    private Enquiry enquiry;

    @ManyToOne
    @JoinColumn(name = "counselor_id", nullable = false)
    private User counselor;

    @Column(nullable = false)
    private LocalDate followupDate;

    @Enumerated(EnumType.STRING)
    private FollowupStatus followupStatus = FollowupStatus.PENDING;

    @Column(columnDefinition = "TEXT")
    private String remarks;

    // Getters, Setters, Constructors
}
```

Table: admissions

```
CREATE TABLE admissions (  
  admission_id INT AUTO_INCREMENT PRIMARY KEY,  
  enquiry_id INT NOT NULL UNIQUE, FOREIGN KEY (enquiry_id) REFERENCES  
enquiries(enquiry_id),  
  student_name VARCHAR(100) NOT NULL,  
  course_id INT NOT NULL, FOREIGN KEY (course_id) REFERENCES  
courses(course_id),  
  total_fees DECIMAL(10,2) NOT NULL,  
  fees_paid DECIMAL(10,2) NOT NULL,  
  payment_status ENUM('Paid','Pending','Installment') NOT NULL,  
  payment_type ENUM('Cash','Online','Cheque','DD') NOT NULL,  
  admission_date DATE DEFAULT (CURRENT_DATE),  
  admitted_by INT, FOREIGN KEY (admitted_by) REFERENCES users(user_id)  
);
```

```
// JPA Entity Equivalent (Spring Boot)  
@Entity  
@Table(name = "admissions")  
public class Admission {  
  @Id  
  @GeneratedValue(strategy = GenerationType.IDENTITY)  
  private Long admissionId;  
  
  @OneToOne  
  @JoinColumn(name = "enquiry_id", unique = true, nullable = false)  
  private Enquiry enquiry;  
  
  @Column(nullable = false, length = 100)  
  private String studentName;  
  
  @ManyToOne  
  @JoinColumn(name = "course_id", nullable = false)  
  private Course course;  
  
  @Column(nullable = false, precision = 10, scale = 2)  
  private BigDecimal totalFees;  
  
  @Column(nullable = false, precision = 10, scale = 2)  
  private BigDecimal feesPaid;  
  
  @Enumerated(EnumType.STRING)  
  private PaymentStatus paymentStatus;  
  
  @Enumerated(EnumType.STRING)  
  private PaymentType paymentType;  
  
  private LocalDate admissionDate = LocalDate.now();  
  
  @ManyToOne  
  @JoinColumn(name = "admitted_by")  
  private User admittedBy;  
  
  // Getters, Setters, Constructors  
}
```

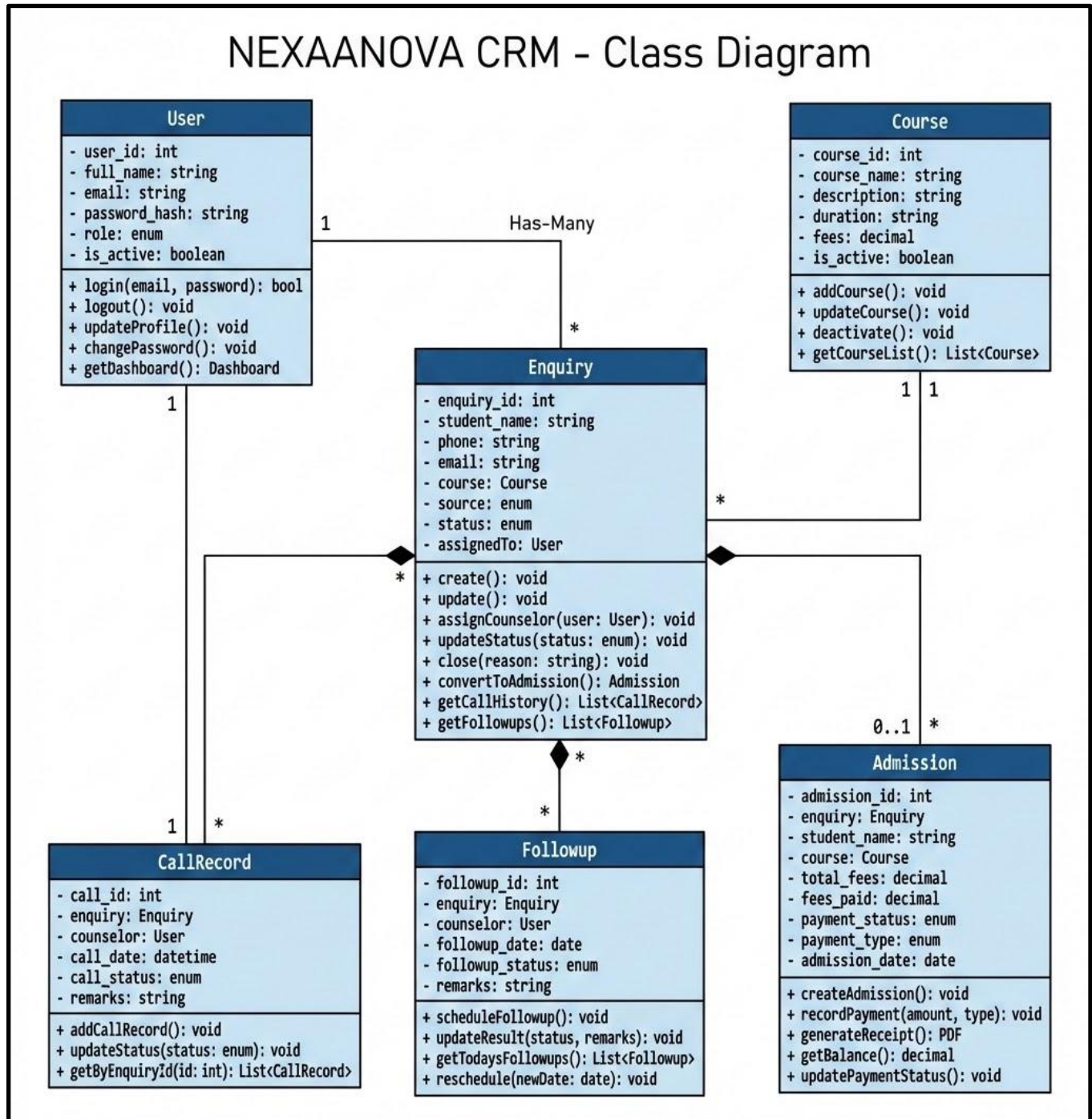
6.3 Indexes for Performance

Index	Table	Purpose
idx_enquiry_assigned	enquiries	Fast lookup by counselor
idx_enquiry_status	enquiries	Filter by pipeline status
idx_followup_date	followups	Today's follow-ups query
idx_call_enquiry	call_records	Call history per enquiry
idx_admission_date	admissions	Date-range reporting

7. CLASS DIAGRAM

The class diagram describes the object-oriented structure of the NEXAANOVA CRM system, showing classes, their attributes, methods, and the relationships between them.

7.1 Core Classes



7.1.1 Class: User

Attributes	Methods
- user_id: int - full_name: string - email: string - password_hash: string - role: enum - is_active: boolean	+ login(email, password): bool + logout(): void + updateProfile(): void + changePassword(): void + getDashboard(): Dashboard

7.1.2 Class: Enquiry

Attributes	Methods
- enquiry_id: int - student_name: string - phone: string - email: string - course: Course - source: enum - status: enum - assignedTo: User	+ create(): void + update(): void + assignCounselor(user: User): void + updateStatus(status: enum): void + close(reason: string): void + convertToAdmission(): Admission + getCallHistory(): List<CallRecord> + getFollowups(): List<Followup>

7.1.3 Class: CallRecord

Attributes	Methods
- call_id: int - enquiry: Enquiry - counselor: User - call_date: datetime - call_status: enum - remarks: string	+ addCallRecord(): void + updateStatus(status: enum): void + getByEnquiry(id: int): List<CallRecord>

7.1.4 Class: Followup

Attributes	Methods
- followup_id: int - enquiry: Enquiry - counselor: User - followup_date: date - followup_status: enum - remarks: string	+ scheduleFollowup(): void + updateResult(status, remarks): void + getTodaysFollowups(): List<Followup> + reschedule(newDate: date): void

7.1.5 Class: Admission

Attributes	Methods
- admission_id: int - enquiry: Enquiry - student_name: string	+ createAdmission(): void + recordPayment(amount, type): void + generateReceipt(): PDF

- course: Course - total_fees: decimal - fees_paid: decimal - payment_status: enum - payment_type: enum - admission_date: date	+ getBalance(): decimal + updatePaymentStatus(): void
---	--

7.1.6 Class: Course

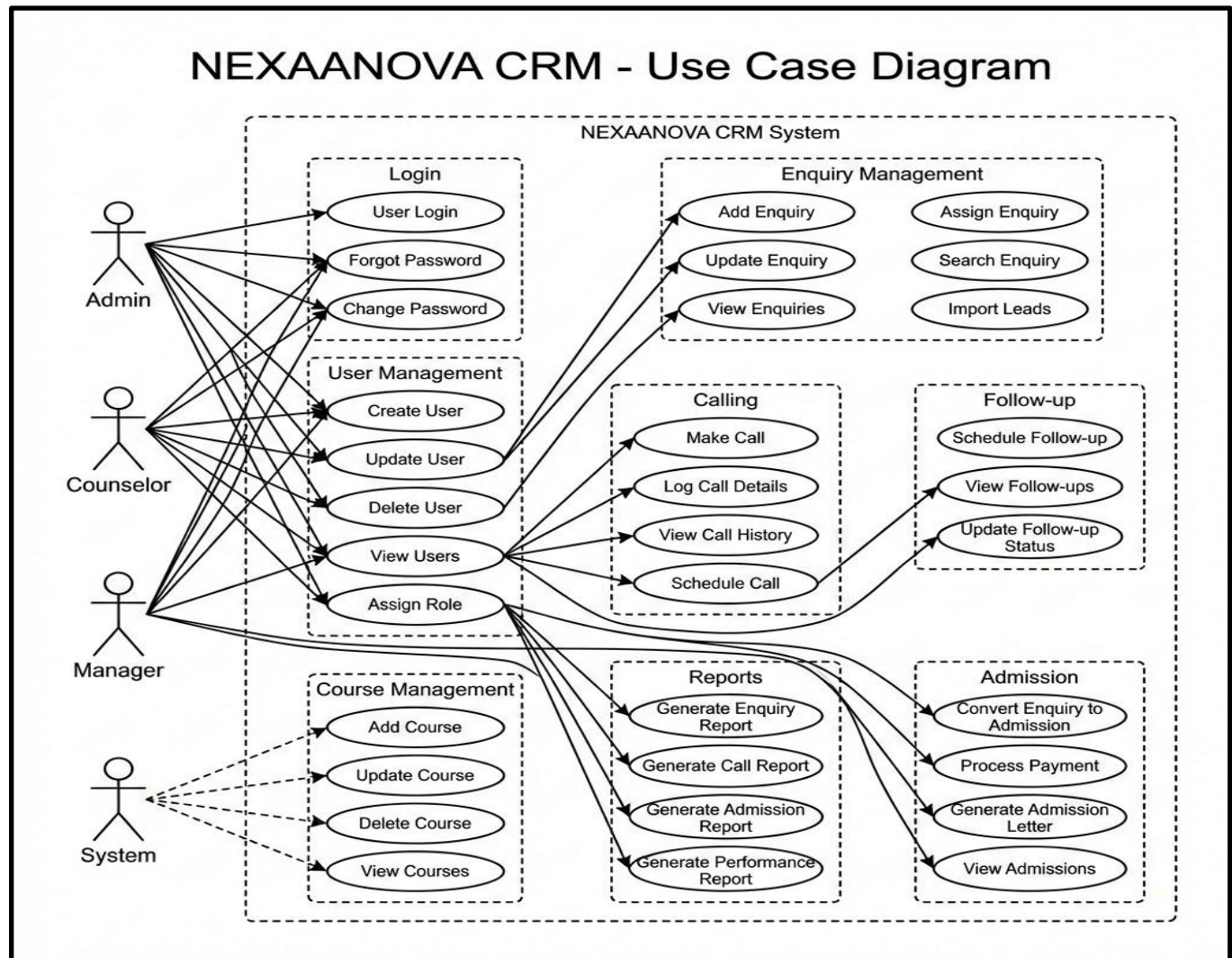
Attributes	Methods
- course_id: int - course_name: string - description: string - duration: string - fees: decimal - is_active: boolean	+ addCourse(): void + updateCourse(): void + deactivate(): void + getCourseList(): List<Course>

7.2 Class Relationships

Class A	Relationship	Class B	Multiplicity
User	Has-Many	Enquiry	1 User → 0..* Enquiries
Enquiry	Has-Many	CallRecord	1 Enquiry → 0..* CallRecords
Enquiry	Has-Many	Followup	1 Enquiry → 0..* Followups
Enquiry	Has-One	Admission	1 Enquiry → 0..1 Admission
Course	Has-Many	Enquiry	1 Course → 0..* Enquiries
Course	Has-Many	Admission	1 Course → 0..* Admissions

8. USE CASE DIAGRAM

Use Case Diagrams describe the functional requirements of the NEXAANOVA CRM system from the perspective of end users. They show the actors (users) and the use cases (actions) they can perform.



8.1 Actors

Actor	Description
Admin	System administrator who manages users, assigns enquiries, and configures the CRM settings.
Counselor	Front-line staff who handle student enquiries, make calls, schedule follow-ups, and process admissions.
Manager	Senior management user who monitors team performance and views reports without data entry rights.
System	Automated actor that sends reminders, assigns IDs, and triggers notifications.

8.2 Use Cases by Module

8.2.1 Login Module

UC#	Use Case	Actor(s)	Precondition
UC01	Login to CRM	Admin, Counselor, Manager	User account exists
UC02	View Role-Based Dashboard	Admin, Counselor, Manager	Logged in
UC03	Logout from System	Admin, Counselor, Manager	Active session exists
UC04	Reset Password	Any User	Account registered

8.2.2 User Management Module

UC#	Use Case	Actor(s)	Precondition
UC05	Create New User	Admin	Admin logged in
UC06	Edit User Details	Admin	User record exists
UC07	Assign Role to User	Admin	User exists
UC08	Activate / Deactivate User	Admin	User record exists
UC09	View User List	Admin	Admin logged in

8.2.3 Enquiry Management Module

UC#	Use Case	Actor(s)	Precondition
UC10	Add New Enquiry	Counselor, Admin	Logged in
UC11	Edit Enquiry Details	Counselor, Admin	Enquiry exists
UC12	Assign Enquiry to Counselor	Admin	Enquiry unassigned
UC13	View Enquiry List	Admin, Counselor	Logged in
UC14	Filter Enquiries by Status/Date	Admin, Counselor	Enquiries exist
UC15	Close Enquiry	Counselor	Enquiry assigned to counselor

8.2.4 Calling Module

UC#	Use Case	Actor(s)	Precondition
UC16	Log Call for Enquiry	Counselor	Enquiry assigned
UC17	Update Call Status	Counselor	Call record exists

UC18	View Call History	Counselor, Admin	Enquiry exists
UC19	Add Call Remarks	Counselor	Call record open

8.2.5 Follow-up Module

UC#	Use Case	Actor(s)	Precondition
UC20	Schedule Follow-up	Counselor	Student marked Interested
UC21	View Today's Follow-ups	Counselor	Follow-ups scheduled
UC22	Update Follow-up Result	Counselor	Follow-up due
UC23	Reschedule Follow-up	Counselor	Follow-up pending
UC24	Receive Follow-up Reminder	System → Counselor	Follow-up date is today

8.2.6 Admission Module

UC#	Use Case	Actor(s)	Precondition
UC25	Convert Enquiry to Admission	Counselor	Student confirmed intent
UC26	Record Payment Details	Counselor	Admission created
UC27	Generate Admission Receipt	System, Counselor	Admission complete
UC28	View Admission Records	Admin, Counselor, Manager	Logged in
UC29	Track Pending Payments	Admin, Manager	Installment set

8.2.7 Course Management Module

UC#	Use Case	Actor(s)	Precondition
UC30	Add New Course	Admin	Admin logged in
UC31	Edit Course Details	Admin	Course exists
UC32	Set / Update Course Fees	Admin	Course exists
UC33	View Course List	Admin, Counselor	Logged in

8.2.8 Reports Module

UC#	Use Case	Actor(s)	Precondition
UC34	View Enquiry Report	Admin, Manager	Logged in
UC35	View Counselor Performance	Admin, Manager	Logged in
UC36	View Follow-up Report	Admin, Manager	Logged in

UC37	View Admission Report	Admin, Manager	Logged in
UC38	Filter Reports by Date Range	Admin, Manager	Report section open
UC39	Export Reports (PDF/Excel)	Admin, Manager	Report generated
UC40	View Conversion Ratio	Admin, Manager	Admissions and enquiries exist

8.3 Use Case Summary

Module	Admin	Counselor	Manager	Total
Login & Access	UC01-04	UC01-04	UC01-04	4
User Management	UC05-09	—	—	5
Enquiry Management	UC10-15	UC10-15	—	6
Calling	UC18	UC16-19	—	4
Follow-up	—	UC20-24	—	5
Admission	UC28	UC25-28	UC28-29	5
Course Management	UC30-33	UC33	—	4
Reports	UC34-40	—	UC34-40	7
TOTAL USE CASES	—	—	—	40